PRACTICAL 7: TRANSFORMER SUMMARIZATION + ROUGE EVALUATION

```
!pip install -q --upgrade --force-reinstall numpy pandas
```

```
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is
google-colab 1.0.0 requires pandas==2.2.3, but you have pandas 3.0.5 which is incompatible.
gradio 6.26.0 requires huggingface-hub<2.0,>=1.16.0, but you have huggingface-hub 0.36.2 which is incompatible.
dask-cudf-cu12 26.2.1 requires pandas<2.4.0,>=2.0, but you have pandas 3.0.5 which is incompatible.
diffusers 0.40.0 requires huggingface-hub<2.0,>=1.23.0, but you have huggingface-hub 0.36.2 which is incompatible.
numba 0.61.2 requires numpy<2.3,>=1.24, but you have numpy 2.5.2 which is incompatible.
cudf-cu12 26.2.1 requires pandas<2.4.0,>=2.0, but you have pandas 3.0.5 which is incompatible.
```

```
# --- Step 0: Install dependencies --
!pip install -q transformers datasets rouge-score evaluate accelerate sumy nltk
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
import numpy as np
import evaluate
from datasets import load_dataset
from transformers import (
    AutoTokenizer, AutoModelForSeq2SeqLM,
    Seq2SeqTrainingArguments, Seq2SeqTrainer,
    DataCollatorForSeq2Seq, pipeline
)
MODEL_NAME = "facebook/bart-large-cnn"   # already pre-trained for summarization
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

```
# --- Step 1: Load tokenizer and pre-trained model --
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForSeq2SeqLM.from_pretrained(MODEL_NAME)
```

```
!pip uninstall -y transformers
!pip install -q "transformers==4.57.1"
```

```
Found existing installation: transformers 4.57.1
Uninstalling transformers-4.57.1:
  Successfully uninstalled transformers-4.57.1
```

```
# --- Step 2: Quick abstractive summary WITHOUT fine-tuning (baseline) --
summarizer = pipeline("summarization", model=model, tokenizer=tokenizer,
                      device=0 if __import__("torch").cuda.is_available() else -1)
sample_article = """
Researchers at a leading university announced a breakthrough in battery technology this week,
unveiling a new solid-state battery design that can charge to 80 percent capacity in under
ten minutes while retaining more than 90 percent of its capacity after 1,000 charge cycles.
The team says the innovation relies on a novel ceramic electrolyte that is both safer and more
energy-dense than the liquid electrolytes used in current lithium-ion batteries. Several major
electric vehicle manufacturers have already expressed interest in licensing the technology,
though the researchers caution that scaling the manufacturing process to an industrial level
could still take several years. The findings were published in a peer-reviewed journal and have
been independently reviewed by other scientists in the field, who called the results promising
but noted that real-world durability testing is still needed before the technology reaches
consumer vehicles.
"""
baseline_summary = summarizer(sample_article, max_length=100, min_length=30, do_sample=False)[0]['summary_text']
print("=== Baseline (no fine-tuning) abstractive summary ===")
print(baseline_summary)
```

```
apacity in under ten minutes. The innovation relies on a novel ceramic electrolyte that is both safer and more energy-dense.
```

```
# --- Step 3: Fine-tune on a small slice of CNN/DailyMail (demonstration) --
raw_dataset = load_dataset("cnn_dailymail", "3.0.0", split="train[:200]")
raw_dataset = raw_dataset.train_test_split(test_size=0.2, seed=42)
MAX_INPUT_LEN = 512
MAX_TARGET_LEN = 64
def preprocess_function(examples):
    inputs = tokenizer(examples["article"], max_length=MAX_INPUT_LEN, truncation=True)
    labels = tokenizer(text_target=examples["highlights"], max_length=MAX_TARGET_LEN, truncation=True)
    inputs["labels"] = labels["input_ids"]
    return inputs
```

```
tokenized_dataset = raw_dataset.map(preprocess_function, batched=True,
                                    remove_columns=raw_dataset["train"].column_names)
data_collator = DataCollatorForSeq2Seq(tokenizer=tokenizer, model=model)
training_args = Seq2SeqTrainingArguments(
    output_dir="./bart-finetuned-summarizer",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=2,
    per_device_eval_batch_size=2,
    weight_decay=0.01,
    save_total_limit=1,
    num_train_epochs=1,          # increase for real fine-tuning
    predict_with_generate=True,
    fp16=True,                   # requires GPU runtime
    logging_steps=10,
    report_to="none",
)
trainer = Seq2SeqTrainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],
    eval_dataset=tokenized_dataset["test"],
    tokenizer=tokenizer,
    data_collator=data_collator,
)
trainer.train()
```

```
Map: 100%                                                40/40 [00:00<00:00, 115.94 examples/s]
/tmp/ipykernel_11839/2620028316.py:28: FutureWarning: `tokenizer` is deprecated and will be removed in version 5.0.0 for `Se
  trainer = Seq2SeqTrainer(
```


[80/80 01:04, Epoch 1/1]

Epoch	Training Loss	Validation Loss
1	1.585000	1.541198

```
/usr/local/lib/python3.13/dist-packages/transformers/modeling_utils.py:3918: UserWarning: Moving the following attributes ir
  warnings.warn(
TrainOutput(global_step=80, training_loss=1.8499454617500306, metrics={'train_runtime': 65.3761,
'train_samples_per_second': 2.447, 'train_steps_per_second': 1.224, 'total_flos': 171510245277696.0, 'train_loss':
1.8499454617500306, 'epoch': 1.0})
```

```
# --- Step 4: Generate summaries with the fine-tuned model --
finetuned_summarizer = pipeline("summarization", model=trainer.model, tokenizer=tokenizer,
                                device=0 if __import__("torch").cuda.is_available() else -1)
test_examples = raw_dataset["test"].select(range(5))
generated_summaries = []
reference_summaries = []
for example in test_examples:
    summary = finetuned_summarizer(example["article"], max_length=64, min_length=20, do_sample=False)[0]['summary_text']
    generated_summaries.append(summary)
    reference_summaries.append(example["highlights"])
print("\n=== Sample generated vs reference summaries ===")
for i in range(2):
    print(f"\n--- Example {i+1} ---")
    print("GENERATED:", generated_summaries[i])
    print("REFERENCE:", reference_summaries[i])
```

```
Device set to use cuda:0

=== Sample generated vs reference summaries ===

--- Example 1 ---
GENERATED: NEW: Congo rebels blame Rwandan Hutu Hutu rebels for attack, report says .
Torrential rain makes it difficult for refugees to move from camp to camp .
U.N. refugee agency says 375,000 Congolese have been forced from their homes in past year .

REFERENCE: NEW: Congo rebels blame Rwandan Hutus for attack, report says .
Internally displaced Congolese flee as rebels attack government troops .
Torrential rain makes refugee movement difficult .
U.N. refugee agency: Some 375,000 have been forced from homes in past year .

--- Example 2 ---
GENERATED: Roma striker Francesco Totti threatens to quit the Italian Players Association .
The Italian Football League has voted to start the next Serie A season on August 26 .
Italy coach Roberto Donadoni wants to play the first round of matches midway through the month .
Totti: "We
REFERENCE: Francesco Totti threatens to quit the Italian Players' Association .
The Italy and Roma striker is upset over the early start to the new season .
He says: We are the principal actors, but ... are never listened to."
```

```
# --- Step 5: Extractive summarization baseline (LexRank) for comparison --
from sumy.parsers.plaintext import PlaintextParser
from sumy.nlp.tokenizers import Tokenizer as SumyTokenizer
```

```
from sumy.summarizers.lex_rank import LexRankSummarizer
def extractive_summary(text, num_sentences=3):
    parser = PlaintextParser.from_string(text, SumyTokenizer("english"))
    summarizer_lr = LexRankSummarizer()
    sentences = summarizer_lr(parser.document, num_sentences)
    return " ".join(str(s) for s in sentences)
print("\n=== Extractive (LexRank) summary of sample article ===")
print(extractive_summary(sample_article, num_sentences=2))
```

```
=== Extractive (LexRank) summary of sample article ===
Researchers at a leading university announced a breakthrough in battery technology this week, unveiling a new solid-state ba
```

```
# --- Step 6: Evaluate abstractive summaries with ROUGE --
rouge = evaluate.load("rouge")
rouge_results = rouge.compute(
    predictions=generated_summaries,
    references=reference_summaries,
    use_stemmer=True
)
print("\n=== ROUGE Scores (fine-tuned model vs reference) ===")
for metric, score in rouge_results.items():
    print(f"{metric}: {score:.4f}")
```

```
=== ROUGE Scores (fine-tuned model vs reference) ===
rouge1: 0.5188
rouge2: 0.2915
rougeL: 0.4074
rougeLsum: 0.5007
```